Inheritance in Java

Inheritance:

- **Inheritance in java** is a mechanism in which one object acquires all the properties and behaviors of parent object.
- The class which inherits the properties of other is known as **subclass** (derived class, child class) and the class whose properties are inherited is known as **superclass** (base class, parent class).
- **extends** is the keyword used to inherit the properties of a class. Following is the syntax of extends keyword.

Inheritance/ IS-A relationship

- Inheritance represents the **IS-A relationship**, also known as *parent-child* relationship.

```
class Animal { }
 class Mammal extends Animal { }
 class Reptile extends Animal { }
class Dog extends Mammal { }
```

- Now, based on the above example, in Object-Oriented terms, the following are true −
- Animal is the superclass of Mammal class.
- Animal is the superclass of Reptile class.
- Mammal and Reptile are subclasses of Animal class.
- Dog is the subclass of both Mammal and Animal classes.
- JAVApoint :

https://www.javatpoint.com/what-is-is-a-relationship-in-java#:~:text=In%20object%2Doriented%20programming%2C%20one,%22is%2Da%22%20relationship.

https://www.javatpoint.com/inheritance-in-java

Inheritance/ IS-A relationship :

- Now, if we consider the IS-A relationship, we can say −
- Mammal IS-A Animal
- Reptile IS-A Animal
- Dog IS-A Mammal
- Hence: Dog IS-A Animal as well

- With the use of the extends keyword, the subclasses will be able to inherit all the properties of the superclass except for the private properties of the superclass.
- We can assure that Mammal is actually an Animal with the use of the instance operator.

Terms used in Inheritance

- **Class:** A class is a group of objects which have common properties. It is a template or blueprint from which objects are created.

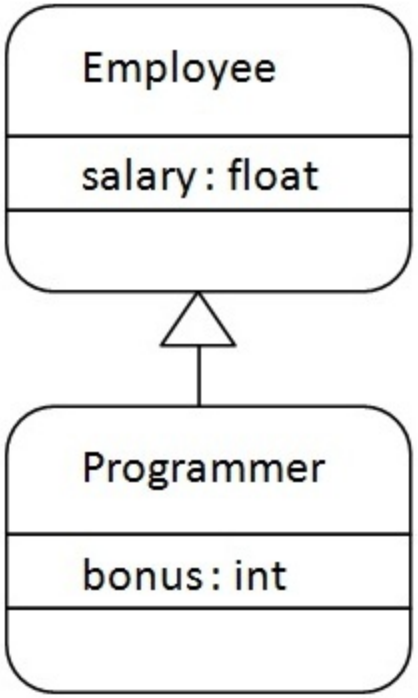
- **Sub Class/Child Class:** Subclass is a class which inherits the other class. It is also called a derived class, extended class, or child class.

- **Super Class/Parent Class:** Superclass is the class from where a subclass inherits the features. It is also called a base class or a parent class.

- **Reusability:** As the name specifies, reusability is a mechanism which facilitates you to reuse the fields and methods of the existing class when you create a new class. You can use the same fields and methods already defined in previous class.

Java Inheritance Example

- Programmer is the subclass and Employee is the superclass.
- Relationship between two classes is **Programmer IS-A Employee**. It means that Programmer is a type of Employee.



Sample Code of Inheritance

```
class Employee
{
 float salary=40000;
}
class Programmer extends Employee
{
int bonus=salary + salary*10%;

  public static void main(String args[])
   {

     Programmer p=new Programmer();

       System.out.println("Programmer salary is:"+p.salary);
       System.out.println("Bonus of Programmer is:"+p.bonus);
   }
}
```

Another Example

```
class Calculation {
  int z;

  public void addition(int x, int y) {
    z = x + y;
    System.out.println("The sum of the given numbers:"+z);
  }

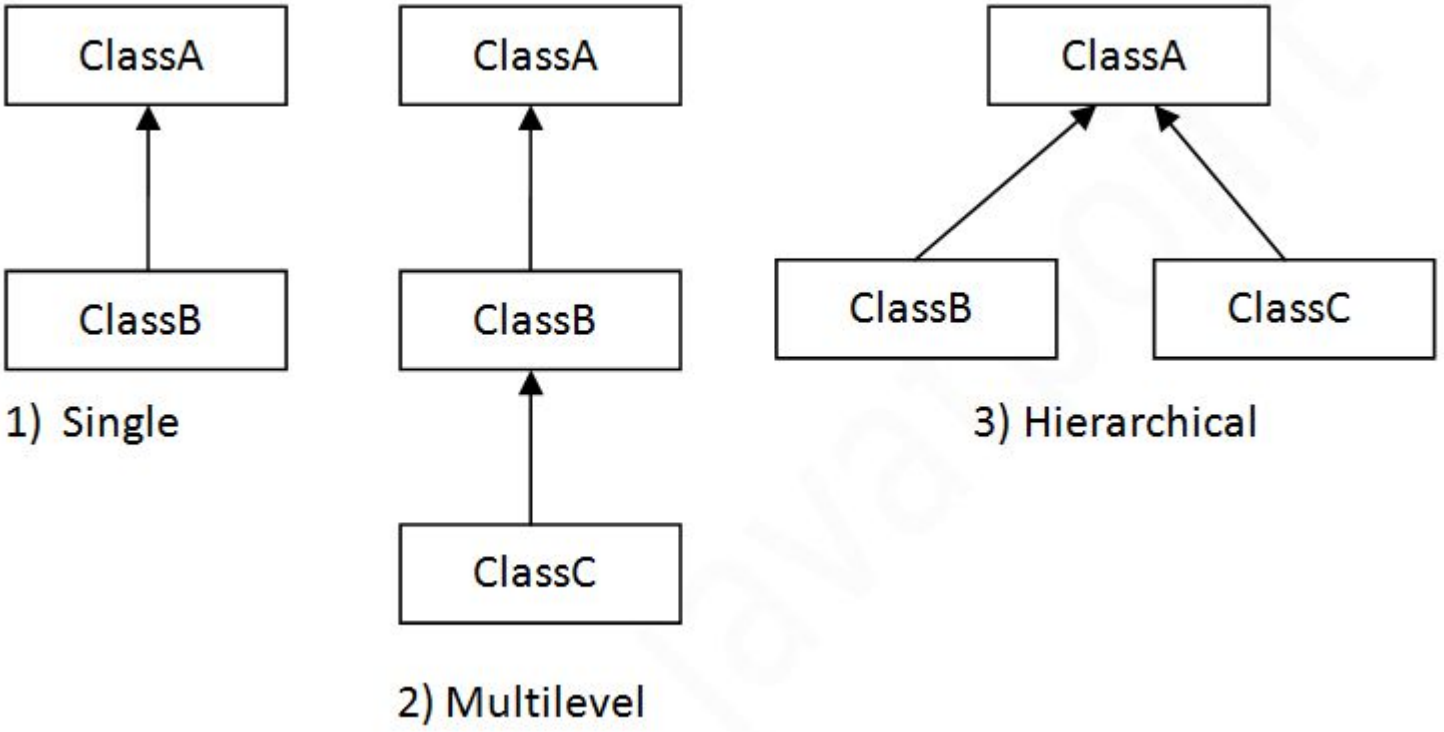
  public void Subtraction(int x, int y) {
    z = x - y;
    System.out.println("The difference between the given numbers:"+z);
  }
}
public class MainClass extends Calculation {
  public void multiplication(int x, int y) {
    z = x * y;
    System.out.println("The product of the given numbers:"+z);
  }
  public static void main(String args[]) {
    int a = 5, b = 10;
    MainClass obj1 = new MainClass();
    obj1.addition(a, b);
    obj1.Subtraction(a, b);
    obj1.multiplication(a, b);
  }
}
```

Output :

The sum of the given numbers:15
The difference between the given numbers: -5
The product of the given numbers: 50

Types of inheritance in java

- There can be three types of inheritance in java: 1. single, 2. multilevel 3. hierarchical.



Single Inheritance Example

```
class Animal
{
void eat(){System.out.println("eating !!!");}
}
class Dog extends Animal
{
void bark(){System.out.println("barking !!!");}
}

class SingleInheritance
{
public static void main(String args[])
 {
  Dog d=new Dog();
  d.bark();
  d.eat();
  }
}
```

Output :

barking !!!
Eating !!!

Multilevel Inheritance Example

```
class Animal
{
void eat(){System.out.println("eating !!!");}
}
class Dog extends Animal
{
void bark(){System.out.println("barking !!!");}
}
class BabyDog extends Dog
{
void weep(){System.out.println("weeping !!!");}
}
class MultilevelInheritance
{   public static void main(String args[])
  {
   BabyDog d=new BabyDog();
   d.weep();
   d.bark();
   d.eat();
  }
}
```

Output :

```
weeping !!!
barking !!!
eating !!!
```

Hierarchical Inheritance Example

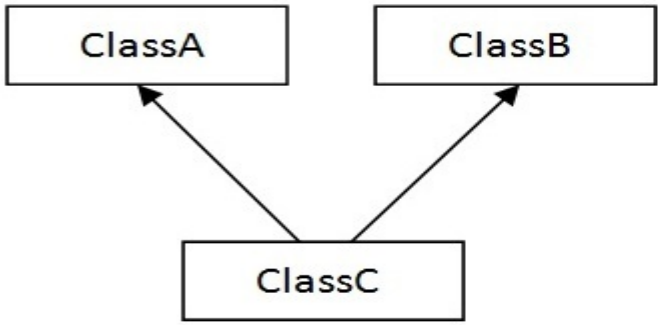
```
class Animal
{
void eat(){System.out.println("eating !!!");}
}
class Dog extends Animal{
void bark(){System.out.println("barking !!!");}
}
class Cat extends Animal{
void meow(){System.out.println("meowing !!!");}
}
class Human extends Animal{
void run(){System.out.println("meowing !!!");}
}
class BabyCat extends Cat{
void meow2(){System.out.println("meowing !!!");}
}
```

Output :
meowing !!!
eating !!!

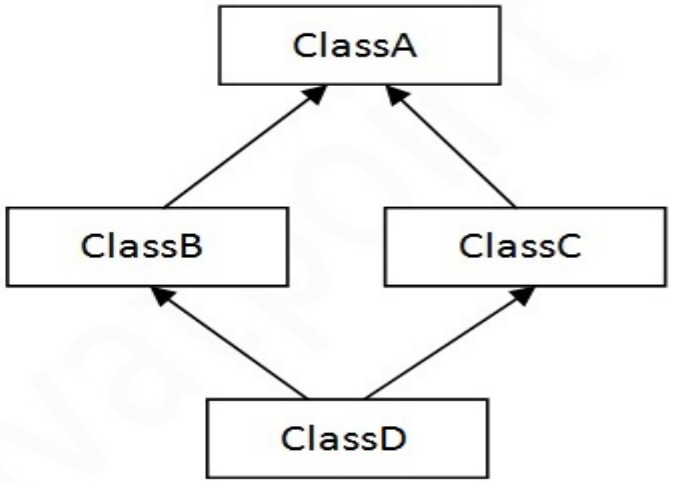
```
class HierarchicalInheritance
{
   public static void main(String args[])
```

Why multiple inheritance is not supported in java?

- Multiple inheritance is not supported in java through class.
- When a class extends multiple classes i.e. known as multiple inheritance.
- To reduce the complexity and simplify the language, multiple inheritance is not supported in java. Such as:



4) Multiple



5) Hybrid

Why multiple inheritance is not supported in java?

- Consider a scenario where A, B and C are three classes. The C class inherits A and B classes. If A and B classes have same method and you call it from child class object, there will be ambiguity to call method of A or B class.

- Since compile time errors are better than runtime errors, java renders compile time error if you inherit 2 classes. So whether you have same method or different, there will be compile time error now.

Why multiple inheritance is not supported in java?

```
class A{
void msg(){System.out.println("Hello");}
}
class B{
void msg(){System.out.println("Welcome");}
}
class C extends A,B{//suppose if it were

 Public Static void main(String args[]){
   C obj=new C();
   obj.msg();//Now which msg() method would be invoked?
}
}
```

Advantages and Disadvantages of Inheritances in Java

Advantages:-

- One of the key benefits of inheritance is to minimize the amount of duplicate code in an application by sharing common code amongst several subclasses. Where equivalent code exists in two related classes, the hierarchy can usually be refactored to move the common code up to a mutual superclass. This also tends to result in a better organization of code and smaller, simpler compilation units.
- Inheritance can also make application code more flexible to change because classes that inherit from a common superclass can be used interchangeably. If the return type of a method is superclass.

Advantages and Disadvantages of Inheritances in Java

- **Reusability** - facility to use public methods of base class without rewriting the same.
- **Extensibility** - extending the base class logic as per business logic of the derived class.
- **Data hiding** - base class can decide to keep some data private so that it cannot be altered by the derived class
- **Overriding** -With inheritance, we will be able to override the methods of the base class so that meaningful implementation of the base class method can be designed in the derived class.

Advantages and Disadvantages of Inheritances in Java

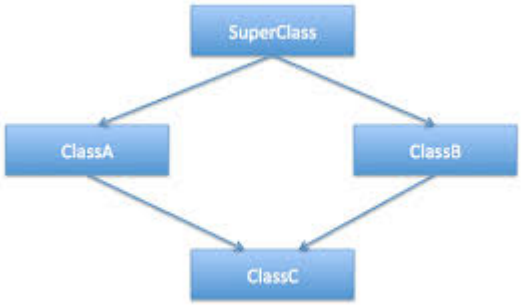
Disadvantages:

- One of the main disadvantages of inheritance in Java (the same in other object-oriented languages) is the increased time/effort it takes the program to jump through all the levels of overloaded classes. If a given class has ten levels of abstraction above it, then it will essentially take ten jumps to run through a function defined in each of those classes

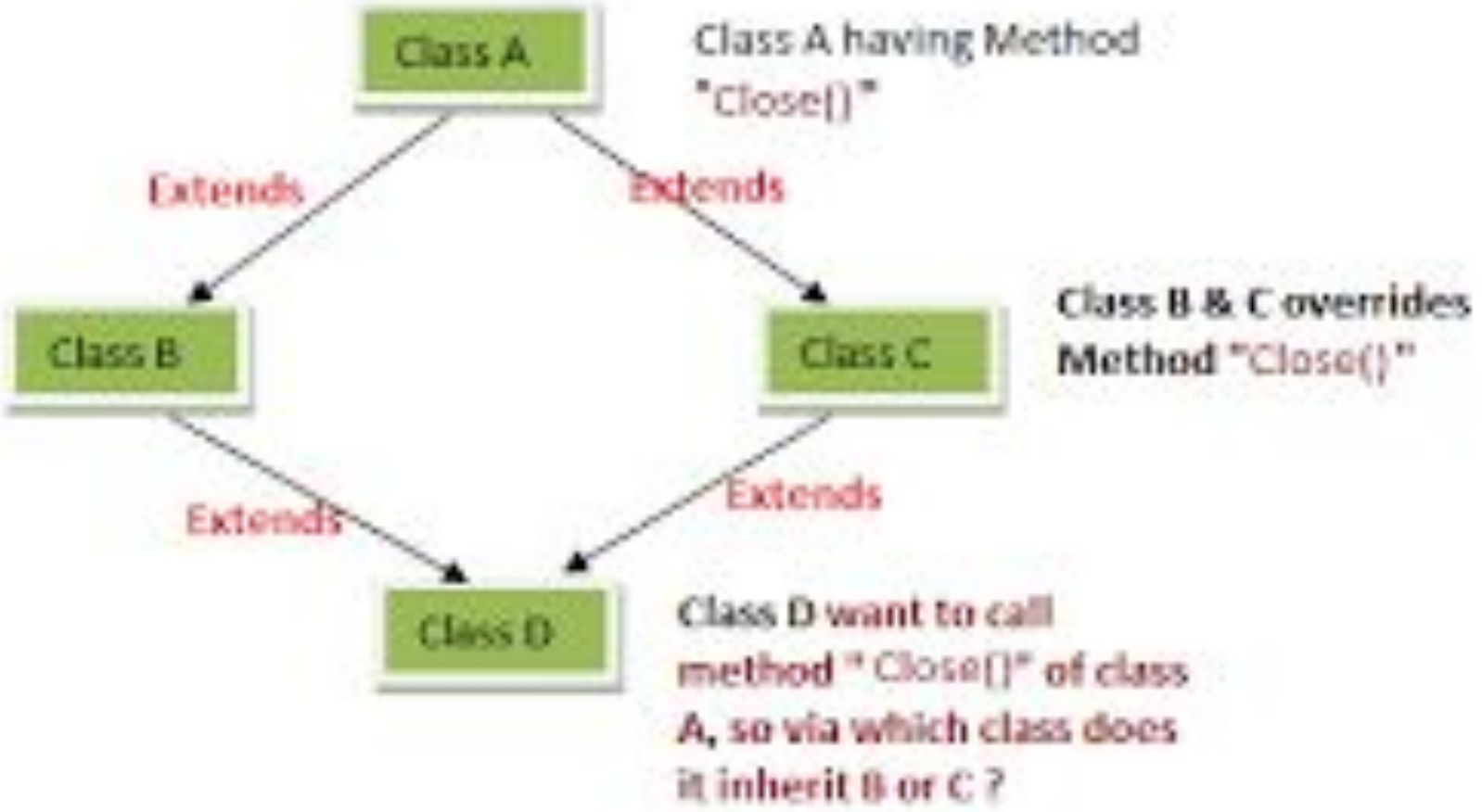
- Main disadvantage of using inheritance is that the two classes (base and inherited class) get tightly coupled. This means one cannot be used independent of each other.

Diamond problem

The Diamond Problem arises when a class inherits from two or more parent classes that have the same method or field. This causes a conflict, as the compiler cannot determine which parent class method or field should be used by the subclass. The diagram below illustrates the Diamond Problem:



```
public class A
 { public void foo()
{ System.out.println("Method foo() from
class A"); }
}
 public class B { public void foo()
{ System.out.println("Method foo() from
class B"); }
} public class C extends A, B
{ public void bar()
{ super.A.foo(); // Call the foo() method
from class A } }
```


Class A
Class A having Method "Close()"
Extends
Extends
Class B
Class C
Class B & C overrides Method "Close()"
Extends
Extends
Class D
Class D want to call method "Close()" of class A, so via which class does it inherit B or C ?

Advantages and Disadvantages of Inheritances in Java

Disadvantages:

- Also with time, during maintenance adding new features both base as well as derived classes are required to be changed. If a method signature is changed then we will be affected in both cases (inheritance & composition)

- If a method is deleted in the "base class" or aggregate, then we will have to re-factor in case of using that method. Here things can get a bit complicated in case of inheritance because our programs will still compile, but the methods of the subclass will no longer be overriding base class methods. These methods will become independent methods in their own right.

Thank You.